

SOAL 1

1) Mengisi Kotak (I-2019-KR-06B)

Andi si berang-berang harus memasukkan bola berbagai ukuran: 5 bola besar, 2 bola ukuran sedang dan 5 bola kecil ke dalam kotak.

- Besar 
- Sedang 
- Kecil 

Sebuah bola hanya akan masuk ke dalam kotak yang lebarnya sama, atau lebih besar. Setiap kotak hanya dapat berisi satu bola. Di dalam sebuah kotak besar, Andi dapat menaruh bola kecil.

Tantangan: Berapa banyak bola yang dapat dimasukkan?

Pilihan Jawaban: a) 9 b) 10 c) 8 d) 11

Kotak yang tersedia adalah 3 kotak besar, 5 kotak sedang, dan 3 kotak kecil.

- Besar 
- Sedang 
- Kecil 

Figure 1: Mengisi Kotak

Observasi

1. Jika kita memerhatikan gambar di soal, jumlah bola yang tersedia 12 buah. Sementara itu, jumlah kotak yang tersedia hanya 11. Artinya jumlah maksimal bola yang kita dapat masukkan hanyalah $X \leq 11$.
2. Kita bisa memasukkan bola dalam kotak dengan 2 strategi utama. Pertama masukan sesuai dengan ukuran mereka, bola kecil akan dimasukkan ke kotak kecil dan seterusnya. Kedua kita menggunakan strategi *Greedy*, dimana saat kita temukan kotak yang lebih besar dari bola yang kita ambil saat ini langsung dimasukkan.

Kalau menggunakan strategi *greedy*, kita bisa memasukkan 10 bola. Jadi kita harus membuat algoritma *greedy*

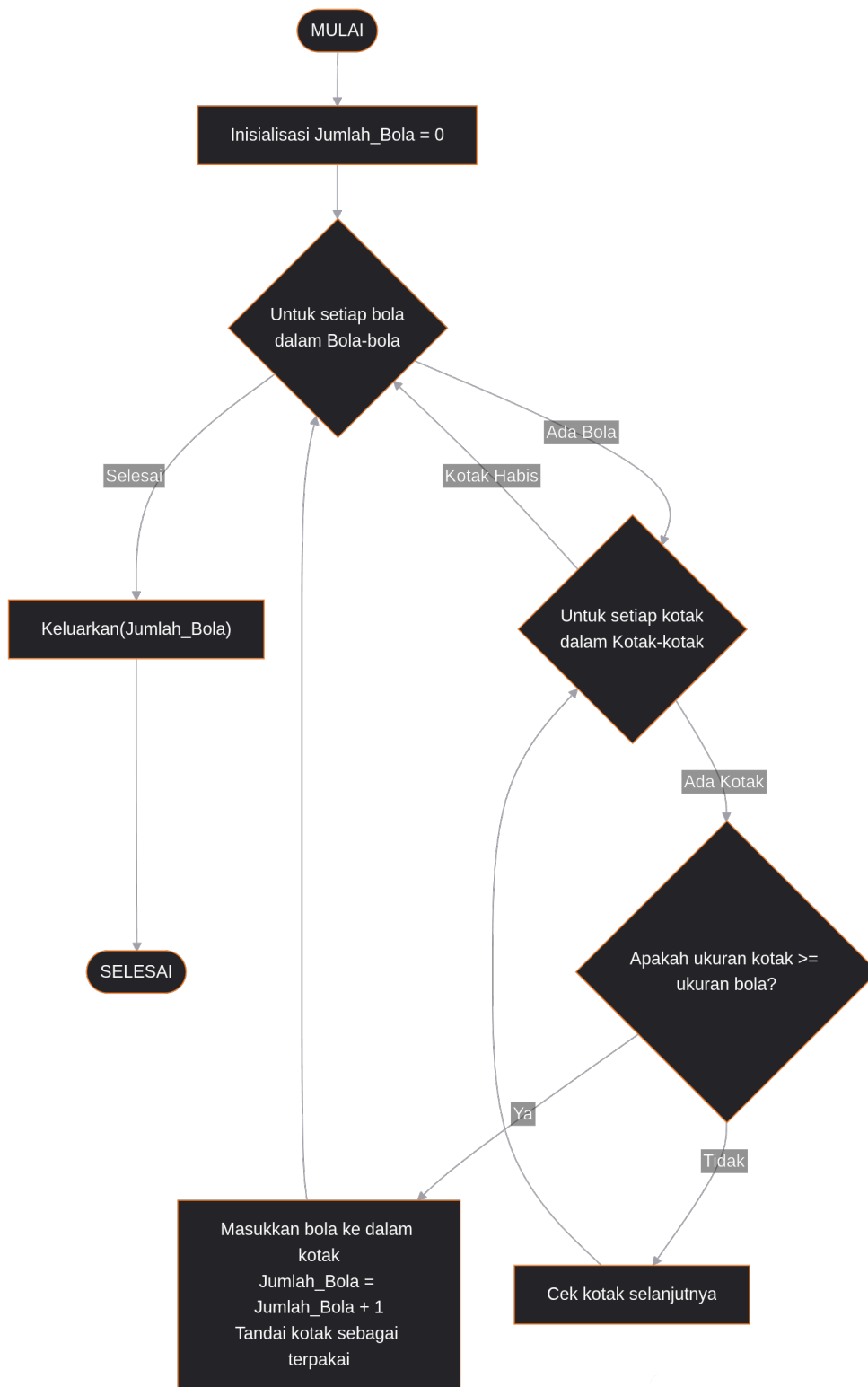
Pseudocode dan Flowchart

```
Mencari_Jumlah_Bola_Maksima(Bola-bola, Kotak-kotak) {
```

```
    Untuk setiap anggota bola dari bola-bola cek setiap kotak dari Kotak-kotak {  
        Jika ukuran kotak  $\geq$  ukuran bola, masukan bola  
        Jumlah bola + 1  
  
        Jika tidak, cek kotak selanjutnya  
    }
```

```
Keluarkan(Jumlah Bola)
```

```
}
```



MermaidEditor.io

Figure 2: Flowchart Soal 1

SOAL 2

2) Kanguru Pelompat (2020-LT-01)

Sisi si Kanguru ingin pulang. Ia dapat melompat sepanjang jalur, secara vertikal (ke atas atau ke bawah), atau mendatar (ke kiri atau ke kanan), yang jalurnya tidak mengandung lebih dari 2 batu bata.

Sisi ingin pulang secepat mungkin.

Tantangan:

Berapa lompatan yang harus dilakukan untuk pulang secepat mungkin?

Jawablah dengan **sebuah bilangan bulat!**

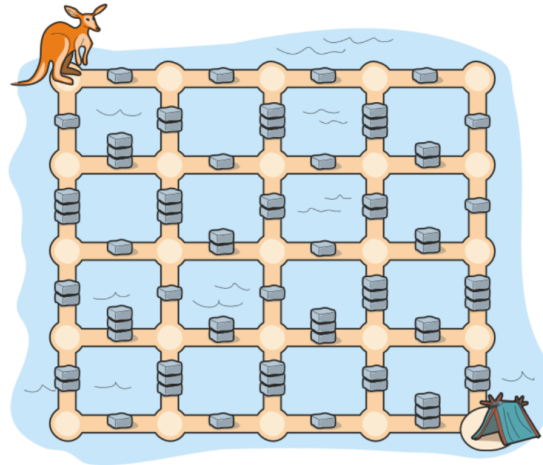


Figure 3: Soal 2

Observasi

Bayangkan jalur yang dapat dilalui oleh kanguru sebagai matriks. Dimana setiap dinding di representasikan dengan angka 0 (tidak ada bata), 1 (1 bata), 2 (2 bata), 3 (3 bata), dan # (air) tidak bisa dilalui

```
0 1 0 1 0 1 0 1 0
1 # 2 # 3 # 3 # 1
0 3 0 1 0 1 0 2 0
3 # 3 # 2 # 2 # 1
0 1 0 2 0 1 0 2 0
2 # 2 # 1 # 3 # 3
0 3 0 2 0 2 0 1 0
3 # 3 # 2 # 2 # 2
0 1 0 1 0 1 0 3 X
```

Dimana X adalah titik akhir.

Kita bisa menemukan jumlah lompatan minimal dengan menggunakan algoritma BFS (Breadth First Search). BFS akan bergerak dari level ke level mengecek semua arah kemudian mengeluarkan langkah minimal. Mengapa kita menggunakan BFS? BFS secara matematika akan mengeluarkan nilai terkecil saat menemukan path dari titik mulai ke titik akhir.

Pseudocode dan Flowchart

```
Cari_Lompatan_Minimal (Matriks Peta Kanguru) {
```

```
    MatriksJarak = Matriks 2D sebesar Matriks Peta Kanguru yang diisi -1
    q = queue kosong
```

```

Masukan titik awal (0, 0) ke q
MatriksJarak[0][0] = 0

ArahLompatHorizontal = [Kiri, Kanan, 0, 0]
ArahLompatVertikal = [0,0, Kiri, Kanan]

Selama q belum kosong {

    Ambil titik saat ini dari q

    Jika X ditemukan {
        keluarkan(MatriksJarak[X saat ini][Y saat ini])
    }

    Untuk i dari 0 sampai 3 {

        X Selanjutnya = X saat ini + ArahLompatHorizontal[i]
        Y Selanjutnya = Y saat ini + ArahLompatVertikal[i]

        Koordinat_Batu_yang_Dilompat_X = X saat ini + (ArahLompatHorizontal[i]/2)
        Koordinat_Batu_yang_Dilompat_Y = Y saat ini + (AraahLompatVertikal[i]/2)

        Jika Matriks Peta Kanguru [Koordinat_Batu_yang_Dilompat_X]
        [Koordinat_Batu_yang_Dilompat_Y] == 1 batu atau 2 batu
        {
            Jika Matriks Peta Kanguru belum dikunjungi
            {
                MatriksJarak[X Selanjutnya][Y Selanjutnya] = MatriksJarak[X saat
                ini][Y saat ini] + 1
            }
        }
    }
}
}
}

```

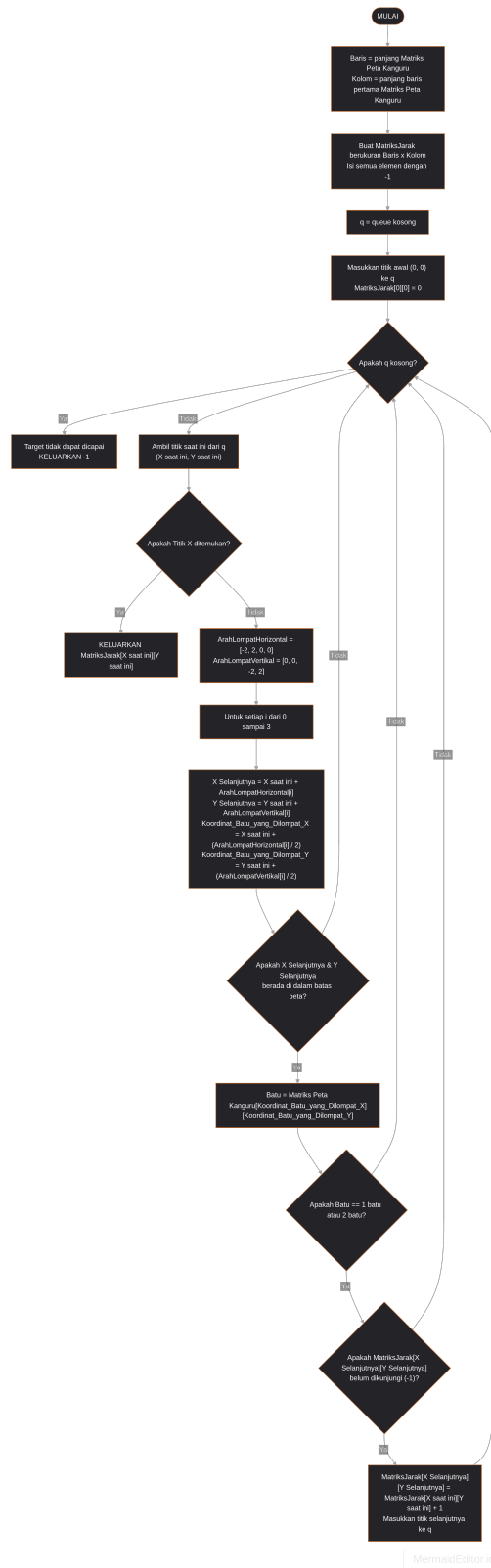
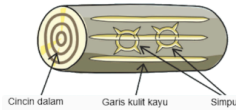


Figure 4: Flowchart Soal 2

SOAL 3

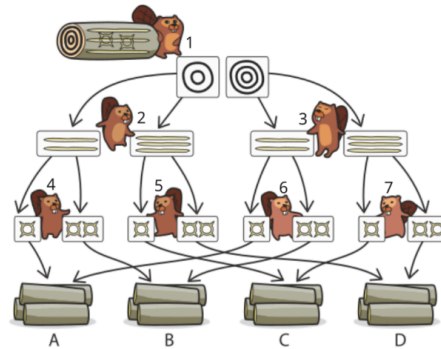
3) Penyortir Balok (2020-PK-02)

Balok kayu di desa Bebras dikelompokkan berdasarkan tiga ciri: jumlah cincin bagian dalam, jumlah garis kulit kayu dan jumlah simpul, seperti pada gambar di bawah ini.



Pada gambar ini, terlihat bagaimana berang-berang menyortir batang kayu berdasarkan cincin bagian dalam, garis kulit kayu, dan simpulnya, dengan meneruskan batang kayu, mulai dari berang-berang paling atas hingga ke tumpukan di bagian bawah.

Batang kayu yang ditunjukkan di atas akan ditempatkan di tumpukan D, karena batang kayu memiliki tiga cincin bagian dalam (dari berang-berang 1 menuju berang-berang 3), tiga garis kulit kayu (dari berang-berang 3 menuju berang-berang 7) dan dua simpul (dari berang-berang 7 menuju tumpukan D).



Tantangan:

Ke tumpukan mana balok ini akan dikirim?

Figure 5: Soal 3

Observasi

Dalam soal ini kita hanya perlu mengkategorikan sebuah input atau objek untuk masuk ke sebuah penampungan. Untuk memudahkan pengerjaan soal, saya akan menggunakan array sebagai penampung benda. Dari sana kita hanya perlu membuat fungsi yang memfilter balok kayu dan memasukkannya ke array benar.

Perhatikan bahwa setiap ciri dari kayu hanya memiliki 2 kemungkinan. Di komputer kita dapat memodelkan ini sebagai true and false. Jadi misalnya 2 cincin dalam menjadi true dan 3 cincin dalam false. Kita hanya perlu membuat 4 array yang memiliki 3 input true atau false untuk setiap ciri.

Asumsikan jenis yang memiliki jumlah paling sedikit dalam kategorinya adalah true. Ex : Cincin 2 (true), Cincin 3 (false)

Maka kita bisa melihat kategorisasi setiap array berdasarkan gambar sebagai berikut :

Cincin Dalam	Garis Kulit	Jumlah Simpul	Array
True	True	True	A
False	False	False	D
True	False	False	D
True	True	False	B
False	True	False	B
True	False	True	C
False	False	True	C
False	True	True	A

Perhatikan bahwa setiap garis kulit dan jumlah simpul yang sama dalam sebuah objek kayu (Ex : GK (true), JS (true)) masuk ke dalam array yang sama. Apa artinya? Cincin dalam tidak berpengaruh sama sekali, maka dari itu kita bisa menyederhanakan tabel menjadi :

Garis Kulit (Bit 1)	Jumlah Simpul (Bit 0)	Binary	Integer	Target Array
False	False	00	0	D
False	True	01	1	C
True	False	10	2	B
True	True	11	3	A

Pseudocode dan Flowchart

```
Penyortir_Balok(Balok Kayu) {
```

```
    Indeks_Array = (BalokKayu.Garis_Kulit di ubah ke biner dan di shift ke kiri) +  
    (BalokKayu.Jumlah_Simpul diubah ke biner)
```

```
    Kategorisasi_Array = [Array D, Array C, Array B, Array A]
```

```
    Masukkan Balok Kayu dalam array Kategorisasi_Array[Indeks_Array]
```

```
}
```

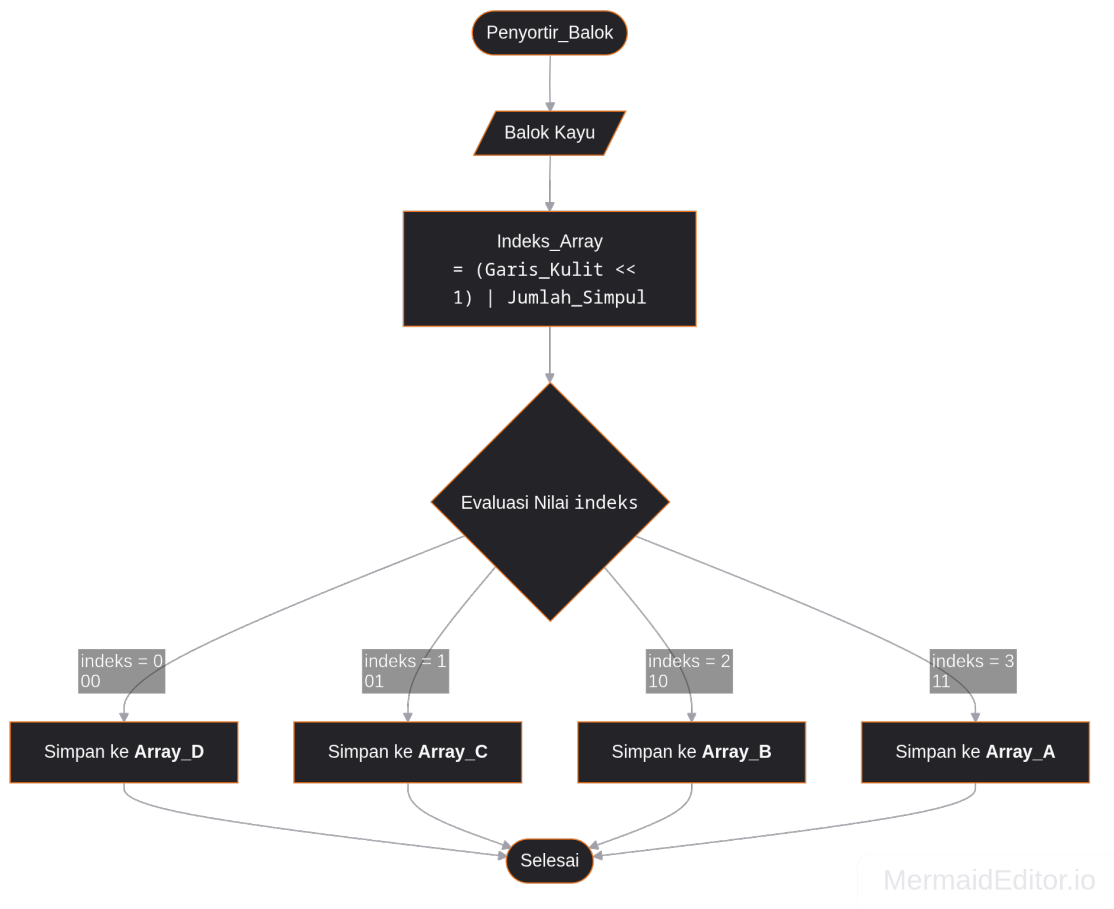


Figure 6: Flowchart Soal 3

SOAL 4

4) Pesan Nenek Moyang (I-2019-CH-10)

Berang-berang si Pintar menemukan sebuah pohon tua yang berlubang. Ternyata, ada pesan rahasia yang diukir pada dinding lubang pohon tersebut. Ternyata, itu adalah kode yang ditulis nenek moyang saat masih hidup di celah tersebut.



Setelah berpikir cukup lama, ia menemukan kunci rahasianya: sebuah huruf abjad adalah kombinasi dari simbol pada judul baris dan kolomnya. Misalnya: huruf H dikode sebagai berikut:

	I	II	III	III	○	○	○	○
☀	A	B	C	D	E	F	G	H
☾	J	K	L	M	N	O	P	Q
☾	S	T	U	V	W	X	Y	Z

$$\text{☀} + \text{○} = \text{☀}$$

Si Pintar menemukan **pesan** sebagai berikut:



Tantangan: Pesan apa yang ditulis nenek moyang?

Pilihan Jawaban: a) LOVEWATER b) SLEEPDAYS c) LOVEMYSUN d) CAREFORME

Figure 7: Soal 4

Observasi

Perhatikan dalam tabel kode susunanannya seperti sebuah matriks. Kita bisa memetakan tabel tersebut menjadi sebuah matriks 3X9. Dalam program kita hanya perlu merepresntasikan setiap string menjadi sebuah karakter misalnya kepala bulat dan rambut runcing adalah A. Gambar kepala adalah koordinat X dan angka adalah Y Setelah itu, kita tinggal memasukan semua input dan mendapat pesan rahasia.

Pseudocode dan Flowchart

Untuk semua gambar dari gambar-gambar {

Translasi_Kode(Gambar_Kepala, Gambar_Angka, Tabel_Kode_Dalam_Matriks,
Tabel_Translasi_Gambar_Ke_Angka) {

Koordinat_X = Tabel_Translasi_Gambar_Ke_Angka[Gambar_Kepala];

Koorinat_Y = Tabel_Translasi_Gambar_Ke_Angka[Gambar_Angka];

Keluarkan(Tabel_Kode_Dalam_Matriks[Koordinat_X][Koorinat_Y])

}

}

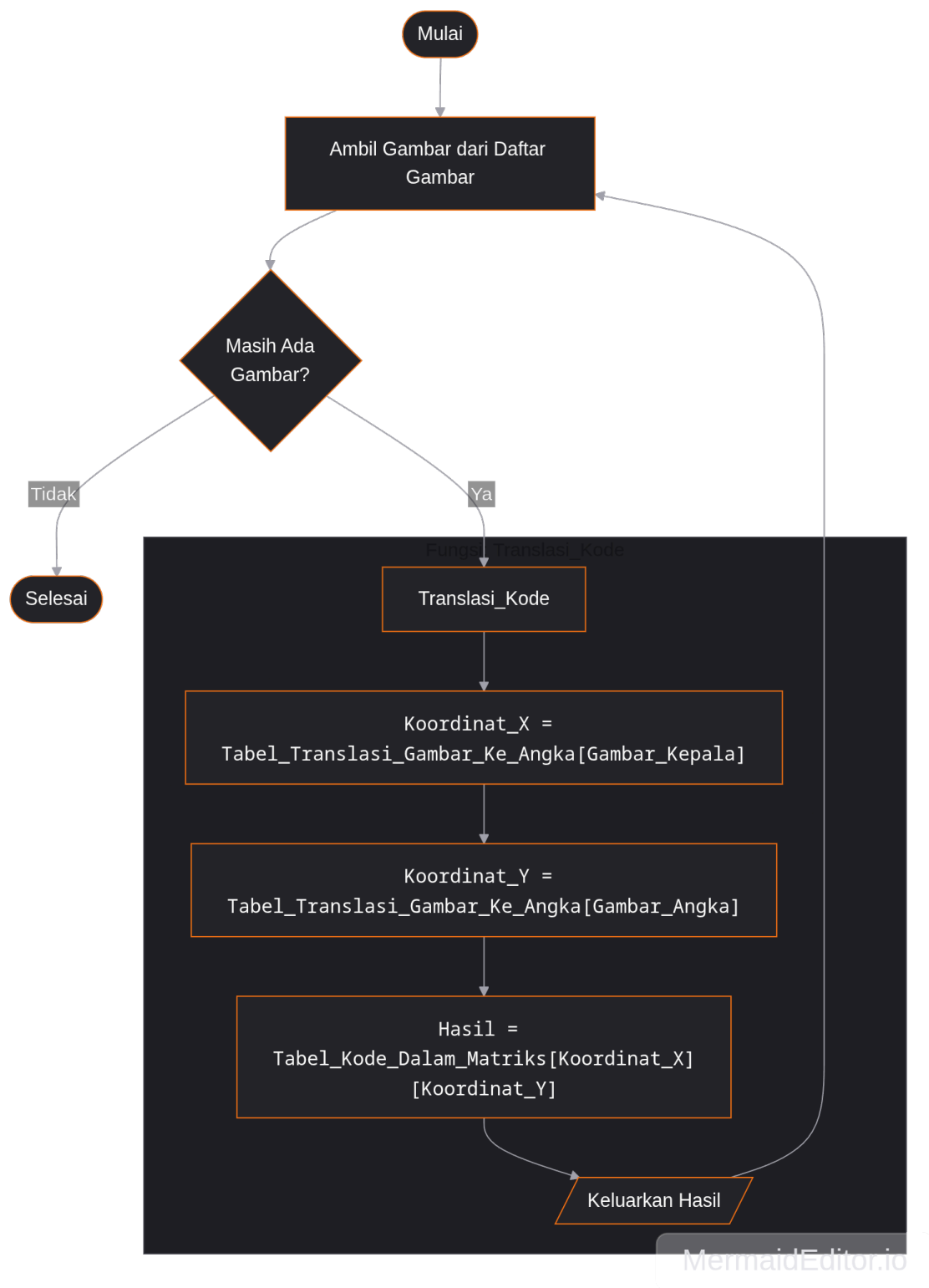


Figure 8: Flowchart Soal 4

Soal 5

5) Penjadwalan Penerbangan (I-2019-IN-03)

Saat sebuah pesawat mendarat di bandara, pendaratan harus dilakukan di sebuah jalur khusus untuk menghindari tabrakan. Dua pesawat hanya boleh mendarat di jalur pendaratan jika selisih waktu kedatangannya lebih dari 15 menit.

Misalnya Jika Penerbangan#1 mendarat pada Pukul 6:10 pagi, Penerbangan #2 mendarat pada Pukul 6:25 pagi and Penerbangan#3 mendarat pada 6:26 pagi, maka Penerbangan #1 dan Penerbangan#2 tidak dapat dijadwal untuk mendarat pada jalur pendaratan yang sama. Sedangkan Penerbangan#3 dan Penerbangan#1 dapat mendarat pada jalur yang sama, tapi tak boleh sama dengan jalur pendaratan Penerbangan#2.

Tantangan: Berapa jumlah minimum jalur pendaratan yang dibutuhkan agar semua pesawat dapat mendarat sesuai jamnya?

Jawab dengan bilangan bulat antara 0 s.d 8.



Anda adalah pengatur pendaratan pesawat hari ini, dan jadwal pendaratan diberikan sebagai berikut:

Nomor penerbangan	Waktu
9W2400	7:00
9W1321	7:21
AI561	7:20
AI620	7:18
EK427	7:03
SG147	7:12

Figure 9: Soal 5

Observasi

1. Untuk menyelesaikan soal ini, kita harus kembali menggunakan algoritma *greedy*, dimana setiap kali ada jalur pesawat yang masih kosong (selang waktu >15 menit), jalur pesawat tersebut langsung digunakan oleh pesawat yang datang. Jalur yang dipakai akan disimpan dalam array Jalur_Pesawat_Terpakai.
2. Kita harus memiliki standar waktu yang sama untuk setiap jam datang. Dalam soal aku akan menghitung jam dengan hitungan setelah tengah malam. Jadi misalnya 07:00 berarti 420 menit setelah tengah malam.

Waktu kedatangan	Ke-	Jalur sawat pakai	Pe- Ter-	Cek Pendaratan yang Ada	Jalur	Status assigned	Tindakan yang Diambil	Jalur sawat pakai Setelah Tindakan	Pe- Ter-
07:00 menit)	(420	[] (Kosong)		Belum ada jalur		Tetap false	Belum ada jalur -> Buka Jalur 1	[420]	
07:03 menit)	(423	[420]		Jalur 1: 423 - 420 = 3 menit (<= 15 menit, jalur masih sibuk)		Tetap false	Semua sibuk -> Buka Jalur 2	[420, 423]	
07:12 menit)	(432	[420, 423]		Jalur 1: 432 - 420 = 12 menit (sibuk) Jalur 2: 432 - 423 = 9 menit (sibuk)		Tetap false	Semua sibuk -> Buka Jalur 3	[420, 423, 432]	
07:18 menit)	(438	[420, 432]	423,	Jalur 1: 438 - 420 = 18 menit (> 15, jalur bebas)		Berubah ke true	Pakai lagi jalur 1 (ubah ke 438)	[438, 423, 432]	
07:20 menit)	(440	[438, 432]	423,	Jalur 1: 440 - 438 = 2 menit (sibuk) Jalur 2: 440 - 423 = 17 menit (> 15, jalur bebas)		Berubah ke true	Pakai lagi jalur 2 (ubah ke 440)	[438, 440, 432]	
07:21 menit)	(441	[438, 432]	440,	Jalur 1: 441 - 438 = 3 menit (sibuk) Jalur 2: 441 - 440 = 1 menit (sibuk) Jalur 3: 441 - 432 = 9 menit (sibuk)		Tetap false	Semua sibuk -> Buka Jalur 4	[438, 440, 432, 441]	

Pseudocode dan Flowchart

```
Ubah_Ke_Menit_dari_Tengah_Malam(waktu) {
    Keluarkan (jam * 60) + menit
}

Hitung_Minimal_Jalur(Daftar_Penerbangan) {

    Daftar_Menit = Array

    Untuk setiap waktu dalam Daftar_Penerbangan {
        Daftar_Menit.tambah(Ubah_Ke_Menit_dari_Tengah_Malam(waktu))
    }
    Urutkan(Daftar_Menit)

    Jalur_Pesawat_Terpakai = Array

    Untuk setiap kedatangan dalam Daftar_Menit {
        ditemukan_jalur = False

        Untuk semua Jalur_Pesawat_Terpakai{
            Jika (kedatangan - Jalur_Pesawat_Terpakai[i] > 15) {
                Jalur_Pesawat_Terpakai[i] = kedatangan
                ditemukan_jalur = True
                break
            }
        }

        Jika Tidak ditemukan_jalur{
            Jalur_Pesawat_Terpakai.tambah(kedatangan)
        }
    }
    Kembalikan Panjang(Jalur_Pesawat_Terpakai)
}
```

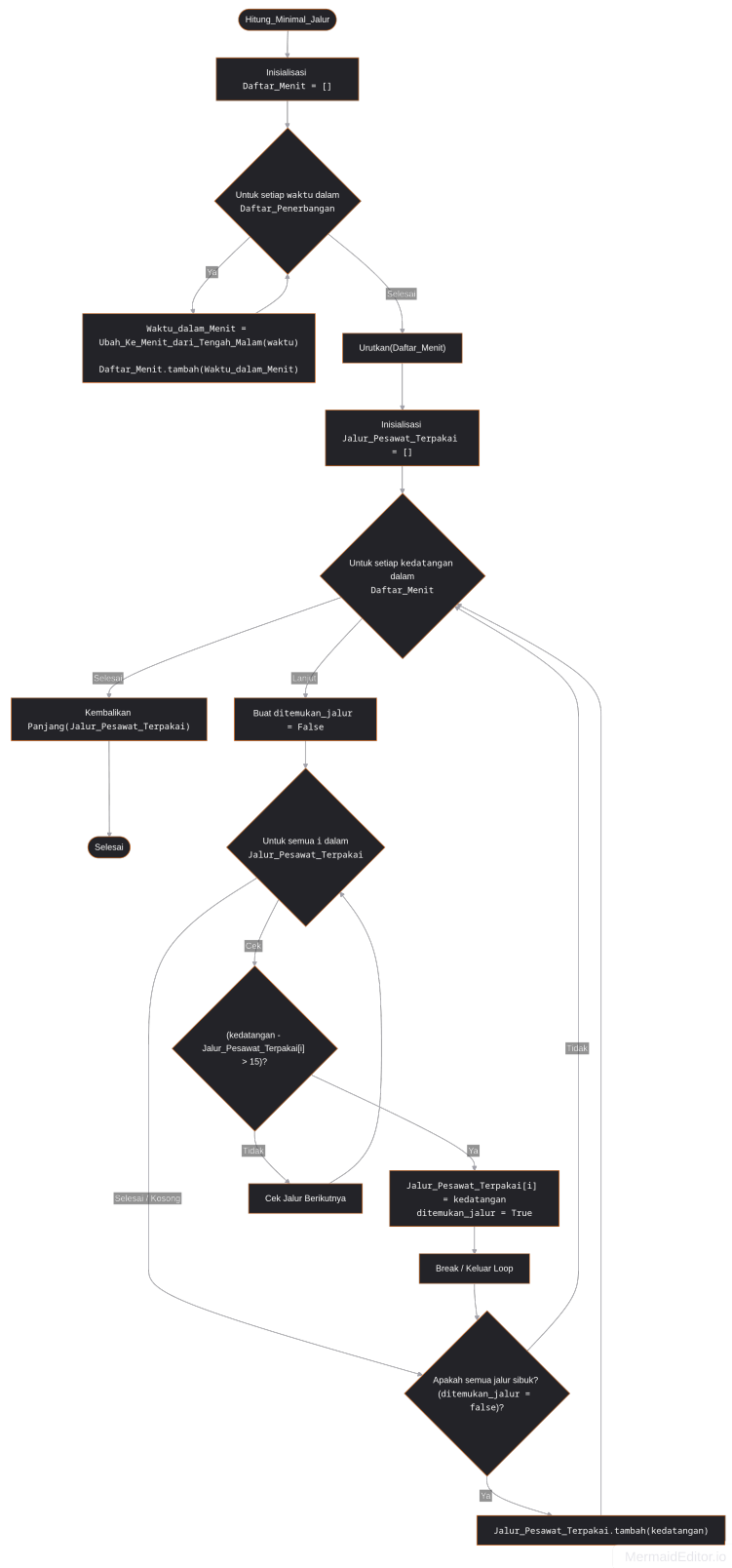


Figure 10: Flow Chart Soal 5